NexusMart - Data Quality Report

Generated 2026-10-17 from **audit.dq_log**, **audit.reject** and the clean tables. Every cleaning decision is implemented in Python (`src/clean/`); nothing was edited by hand in Excel or the database. In dq_log, **before** = records (or distinct label spellings) affected before the fix; **after** = records still unresolved after the fix.

1. Rows read, rejected and loaded per table

Table	Rows read (raw)	- Exact duplicate rows	- Duplicate entities merged	- Rejected (reject table)	= Rows loaded (clean)	Reconciles?	Reject reasons
customers	45,000	0	2,556	0	42,444	OK	-
products	3,200	0	26	0	3,174	OK	-
sellers	420	0	0	0	420	OK	-
locations	650	0	0	0	650	OK	-
orders	163,200	3,200	0	13	159,987	OK	BAD_DATE 11, DATE_OUT_OF_RANGE 2
order_items	315,143	0	0	40	315,103	OK	PARENT_ORDER_REJECTED 31, BAD_QTY 9
payments	168,111	3,296	0	13	164,802	OK	PARENT_ORDER_REJECTED 13
shipping	151,228	0	0	6	151,222	OK	PARENT_ORDER_REJECTED 6
returns	16,048	0	0	140	15,908	OK	ORPHAN_RETURN 139, PARENT_ORDER_REJECTED 1

How to read it: rows read - exact duplicate rows - duplicate entities merged - rejected = rows loaded. Example, orders: 163,200 - 3,200 - 0 - 13 = 159,987. Rejected records are never deleted silently: each sits in the reject table with a reason code and the full raw row. Child rows of a rejected order (PARENT_ORDER_REJECTED) are rejected with it so no orphan child remains.

2. Cleaning decisions with before / after counts

Table	Issue	Column	Before	After	Fix applied	Assumption / business reason
locations	BAD COORDINATES	latitude/longitude	68	62	6 swapped lat/lon fixed; 62 outside Pakistan set to NULL	Pakistan bbox lat 23-38, lon 60-78
locations	MISSING PROVINCE	province	25	0	filled from 5 nearest cities by coordinates	leave-one-out accuracy 87.0%; applied only if >= 85%
sellers	RATING OUT OF RANGE	rating	22	0	set to NULL	valid rating 1.0-5.0; 33 already missing stay NULL (not imputed: ratings are a quality signal)
products	MISSING BRAND	brand	392	10	filled from first word of product_name when it is a known brand; else 'Unknown'	product names start with the brand in this catalogue
products	MISSING SUB_CATEGORY	sub_category	485	10	inferred from (category, product type); remainder 'Unknown'	475 inferred where the type->sub_category mapping is >=90% consistent
products	BAD WEIGHT	weight_kg	17	0	set to NULL	valid 0<w<=100; 296 originally missing stay NULL
products	BAD LIST PRICE	list_price	13	0	replaced with median unit_price actually charged on that product's lines	invalid = <=0, sentinel, or >25% away from median charged price (needs >=3 lines)
products	MISSING/BAD COST PRICE	cost_price	237	0	imputed list_price x median(cost/list) of the category; flagged cost_imputed=1	missing 170, cost>list 67; median ratio 0.61
products	DUPLICATE PRODUCTS	product_id	3,200	3,174	merged to lowest product_id; dup->master kept in product_map	match = normalised name + brand + category; sellers/price may differ between listings (order lines keep their own seller_id)
customers	PHONE NORMALISATION	phone	41,401	41,401	digits only, 03XXXXXXXXX format; unparseable -> NULL	3599 originally missing
customers	MISSING / IMPOSSIBLE REGISTRATION DATE	registration_date	319	319	kept as NULL (not repairable from other data)	registration dates legitimately precede the 2024-25 order window
customers	BIRTH YEAR OUT OF RANGE	birth_year	142	0	set to NULL	valid 1940-2010; 4600 originally missing stay NULL
customers	DUPLICATE CUSTOMERS	customer_id	45,000	42,444	merged to lowest customer_id; every dup->master pair kept in customer_map	link = same normalised email OR same phone (phone links also need name similarity >= 0.7); 1 phone links skipped; never name alone
orders	EXACT DUPLICATE ROWS	(all columns)	163,200	160,000	drop_duplicates, count logged	rows identical in every column are the same record exported twice
orders	IMPOSSIBLE / UNPARSEABLE DATE	order_date	1,124	0	see repair below	30 Feb, 31 Apr, month 13, 00/00/yyyy cannot be parsed
orders	DATE OUT OF RANGE	order_date	481	0	see repair below	valid window 2024-01-01..2025-06-30
orders	ORDER DATE REPAIR	order_date	1,605	13	1455 repaired as shipped_date - 1d, 137 as payment_date - 1d, 13 rejected (BAD_DATE / DATE_OUT_OF_RANGE)	shipped_date - order_date is 1 day (0-2 on 100% of valid rows) so it is the most accurate anchor; payment lag is spread over 0-11+ days so it is the fallback
orders	BAD ORDER TOTAL	order_total	488	0	set NULL then recomputed from clean order lines (sentinel 9999999, <=0, 'Rs. 1,234' text parsed)	
orders	BROKEN REFERENCE	customer_id	321	321	kept; flagged orphan_customer=1 -> mapped to Unknown (-1) in DimCustomer	line is never deleted
orders	MISSING / BAD SHIP CITY	ship_city_id	6,388	6,388	left NULL -> Unknown (-1) in DimLocation	NOT filled from the customer's city: ship city differs from customer city on ~11% of orders
order_items	EXACT DUPLICATE ROWS	(all columns)	315,143	315,143	drop_duplicates	
order_items	BROKEN REFERENCE	product_id	932	932	kept; flagged orphan_product=1 -> Unknown (-1) in DimProduct (COGS unknown for these lines)	

Table	Issue	Column	Before	After	Fix applied	Assumption / business reason
order_items	BROKEN REFERENCE	seller_id	0	0	kept; flagged orphan_seller=1 -> Unknown (-1) in DimSeller	
order_items	QUANTITY OUTLIER / INVALID	quantity	948	9	negative -> abs (307); words/'2.0' -> number; 500 / >10 recovered from order_total (632); unrecoverable -> reject BAD_QTY (9)	abs() of negatives matches order_total on ~99.7% of lines; qty 500 is an injected outlier (legit max is 6)
order_items	BAD UNIT PRICE	unit_price	1,557	0	1528 solved from order_total, 29 set to list_price x 1.0259, 0 rejected BAD_PRICE	invalid = <=0, sentinel 9999999, or unit/list ratio outside 0.5-2.0 (valid data sits in 0.99-1.10)
order_items	MISSING / WRONG DISCOUNT	discount_amount	32,446	0	recomputed = round(quantity x unit_price x discount_code%)	31563 blank; 883 disagreed with the code (inflated, > gross); order-level code applies to every line (spec)
orders	ORDER TOTAL vs LINES	order_total	159,499	159,457	order_total filled from sum(line net) where invalid; match rate after repair shown in after_count	before = orders with a valid raw total, after = those whose total equals sum(qty x price - discount) +-2
order_items	NO PRODUCT COST FOR LINE	unit_cost	932	0	unit_cost = unit_price x 0.5719 (median cost/unit_price of lines with a genuine product cost); flagged cost_source=NO_PRODUCT_COST_ESTIMATE	orphan-product lines have no product record; COGS 0 would overstate margin. CostAmount in FactSales = unit_cost x quantity
payments	EXACT DUPLICATE ROWS	(all columns)	168,111	164,815	drop_duplicates	
payments	BAD PAYMENT AMOUNT	amount	985	0	replaced with the order's total (payment amount equals order_total in normal rows)	left NULL where the order total itself is unknown
payments	SPLIT PAYMENTS	order_id	9,630	159,987	orders with 2 payment rows are split payments (parts sum to order_total); is_primary=1 marks one row per order for FactSales	primary = Paid/Refunded first, then larger part, then later payment_date; 55 bad part amounts rebuilt as order_total - other part
payments	MISSING / IMPOSSIBLE DATE	payment_date	5,984	5,984	kept NULL	payment_date is not used for revenue; used only to pick between 2 payment rows
shipping	EXACT DUPLICATE ROWS	(all columns)	151,228	151,228	drop_duplicates	
shipping	MISSING DATE	shipped_date	6,961	6,961	kept NULL	ShippingDays then unknown for that order
shipping	DELIVERED BEFORE SHIPPED	delivered_date	1,269	0	delivered_date set to NULL (flag date_fix)	all such rows are exactly shipped-2 days: a systematic export fault, true date unknown
shipping	DELIVERY > 60 DAYS	delivered_date	266	0	delivered_date set to NULL (flag date_fix)	legit tail ends ~30 days; the 88-102 day cluster is corrupt
shipping	MISSING / BAD SHIPPING FEE	shipping_fee	9,598	0	imputed with the mean valid fee (545); flagged fee_imputed=1	454 sentinel/negative values + 9144 blanks; mean (not median) so total shipping cost is preserved; fee does not depend on courier
returns	EXACT DUPLICATE ROWS	(all columns)	16,048	16,048	drop_duplicates	
returns	BROKEN REFERENCE	order_item_id	139	0	rejected ORPHAN_RETURN	
returns	BAD REFUND AMOUNT	refund_amount	203	0	valid returns with refund <=0 or > line gross set to the line's gross amount (full refund)	refund == line gross in the modal case (7.6k rows); partial refunds (25-99%) are real and untouched. Rejected returns are not valid, amount kept as is
returns	BAD RETURN DATE	return_date	256	0	set to order_date + 10 days (median observed lag); flagged date_fixed=1	invalid = unparseable, before the order date, or >90 days after it

3. Label standardisation (one concept, one spelling)

Table	Column	Raw distinct values (as exported)	Spellings after trimming spaces	Canonical after	Notes
locations	province	22	20	8	0 rows matched fuzzily; 0 unmatched -> 'Unknown'; 25 blank -> 'Unknown'
locations	population_tier	3	3	3	0 rows matched fuzzily; 0 unmatched -> 'Unknown'; 0 blank -> 'Unknown'
sellers	seller_tier	11	8	4	0 rows matched fuzzily; 0 unmatched -> 'Unknown'; 22 blank -> 'Unknown'
products	category	74	62	12	0 rows matched fuzzily; 0 unmatched -> 'Unknown'; 0 blank -> 'Unknown'
customers	gender	9	8	3	0 rows matched fuzzily; 0 unmatched -> 'Unknown'; 2199 blank -> 'Unknown'
orders	order_status	20	19	4	0 rows matched fuzzily; 0 unmatched -> 'Unknown'; 0 blank -> 'Unknown'
orders	channel	12	12	3	0 rows matched fuzzily; 0 unmatched -> 'Unknown'; 0 blank -> 'Unknown'
payments	payment_method	22	21	5	0 rows matched fuzzily; 0 unmatched -> 'Unknown'; 0 blank -> 'Unknown'
payments	payment_status	17	16	4	0 rows matched fuzzily; 0 unmatched -> 'Unknown'; 0 blank -> 'Unknown'
shipping	courier	22	22	6	0 rows matched fuzzily; 0 unmatched -> 'Unknown'; 0 blank -> 'Unknown'
shipping	delivery_status	15	15	3	0 rows matched fuzzily; 0 unmatched -> 'Unknown'; 0 blank -> 'Unknown'
returns	return_status	10	10	3	0 rows matched fuzzily; 0 unmatched -> 'Unknown'; 0 blank -> 'Unknown'
returns	return_reason	38	37	10	0 rows matched fuzzily; 0 unmatched -> 'Unknown'; 2367 blank -> 'Unknown'

Why two 'before' numbers: 'Raw distinct values' counts every distinct string exactly as exported, so 'Fashion' and 'Fashion ' (trailing space) count twice and a blank counts as one value (example: product categories 74 raw -> 62 after trimming spaces -> 12 canonical). The decision log works on trimmed, non-blank values, hence the smaller second number. Case and punctuation variants ('FASHION', 'Home & Kitchen' vs 'Home and Kitchen') are collapsed by the alias dictionary in config.LABELS.

No unmatched labels in this run: every spelling in the supplied data maps to a canonical value. A new spelling in a later data drop is logged here as UNMATCHED LABELS (fuzzy match first), never crashes the pipeline.

4. Reject table: reason codes

Table	Reason code	Records	Meaning
order_items	BAD_QTY	9	quantity invalid and not recoverable from order_total
order_items	PARENT_ORDER_REJECTED	31	child record of an order that was rejected
orders	BAD_DATE	11	order date impossible/unparseable and not repairable from shipping/payment dates
orders	DATE_OUT_OF_RANGE	2	order date outside 2024-01-01..2025-06-30 and not repairable
payments	PARENT_ORDER_REJECTED	13	child record of an order that was rejected
returns	ORPHAN_RETURN	139	return references an order_item that does not exist
returns	PARENT_ORDER_REJECTED	1	child record of an order that was rejected
shipping	PARENT_ORDER_REJECTED	6	child record of an order that was rejected

Most rejects are child records of the 13 orders whose date could not be repaired (PARENT_ORDER_REJECTED) and the 139 returns pointing at order lines that do not exist (ORPHAN_RETURN). Orphan customers/products are **not** rejected: the line is kept and mapped to the Unknown member.

5. Sensitivity: how much does each big decision move the headline numbers?

Answer-key tolerance is about +-1% for revenue and +-0.5 percentage points for rates. Each row recomputes the metric with the alternative treatment. Only decisions marked YES move a headline number beyond tolerance; they are documented as key assumptions.

Alternative treatment tested	Rows affected	Metric	Our baseline	Alternative	Change	> tolerance?
Price repair -> reject repaired lines instead	1,557	NMV	1,199,960,496	1,194,357,564	-0.467 %	no
Price repair -> list price instead of order_total	1,528	NMV	1,199,960,496	1,199,738,559	-0.018 %	no
Qty outlier (500) -> reject instead of recover from order_total	632	NMV	1,199,960,496	1,197,685,082	-0.190 %	no
Negative qty -> drop lines instead of abs()	307	NMV	1,199,960,496	1,198,713,646	-0.104 %	no
Order date repair -> reject those orders instead	1,592	NMV	1,199,960,496	1,187,626,162	-1.028 %	YES
Discount -> trust raw discount_amount (blank=0) instead of code%	31,563	NMV	1,199,960,496	1,206,131,656	+0.514 %	no
Orphan-product lines -> drop instead of Unknown member	932	NMV	1,199,960,496	1,195,663,078	-0.358 %	no
In Transit excluded from NMV (spec says INCLUDE)	2,692	NMV	1,199,960,496	1,188,945,224	-0.918 %	no
Refund repair -> leave raw refund (0 / > line gross)	203	NetRevenue	1,142,292,237	1,141,927,397	-0.032 %	no
Duplicate orders NOT removed (cancellation rate)	3,213	CancelRate	9.16	9.17	+0.014 pts	no
Rejected returns counted as valid (spec says NOT valid)	1,311	ReturnRate	5.15	5.61	+0.462 pts	no
Order date repair by payment_date instead of shipped_date (monthly shift only; NMV total unchanged)	122	NMV moved between months			+0.069 % of NMV	no
Imputed cost -> exclude those products from margin instead	21,625	MarginPct	26.69	26.70	+0.013 pts	no
Imputed cost biased by -5% (about the median imputation error)	21,625	MarginPct	26.69	26.95	+0.264 pts	no
Imputed cost biased by +5% (about the median imputation error)	21,625	MarginPct	26.69	26.42	-0.264 pts	no
Imputed cost biased by -20% (stress test)	21,625	MarginPct	26.69	27.74	+1.054 pts	stress only
Imputed cost biased by +20% (stress test)	21,625	MarginPct	26.69	25.63	-1.054 pts	stress only
cost_price > list_price kept as valid instead of imputed	6,772	MarginPct	26.69	24.69	-2.000 pts	YES
Orphan-product lines: COGS = 0 instead of estimated cost	932	MarginPct	26.69	26.91	+0.220 pts	no
Orphan-product lines: excluded from margin instead	932	MarginPct	26.69	26.64	-0.042 pts	no
Customer dedupe (active customers)	2,556	ActiveCustomers	32,836	31,779	-3.219 %	YES

Beyond tolerance (document as key assumptions in the README): Order date repair -> reject those orders instead (-1.03 % on NMV); cost_price > list_price kept as valid instead of imputed (-2.00 pts on MarginPct); Customer dedupe (active customers) (-3.22 % on ActiveCustomers).

6. Validation of the clean output

Check	Result
order_total equals sum(qty x price - discount) +-2 (valid raw totals)	99.97%
Primary keys unique (orders, items, payments, shipments, returns, customers, products)	PASS
order_items.order_id / payments / shipping -> orders	PASS
returns.order_item_id -> order_items	PASS
order_items.seller_id -> sellers; ship_city_id -> locations	PASS
Orphans flagged, not dropped (customer / product lines)	321 orders / 932 lines -> Unknown member
All order dates inside 2024-01-01..2025-06-30	PASS
Quantity 1..10 and unit_price > 0 on every kept line	PASS
Distinct canonical categories / payment methods / couriers	12 / 5 / 6
Split-payment orders (2 payment rows): parts add up to order_total (+-2)	4,815 orders, 96.7% add up
Exactly one is_primary payment row per order (FactSales picks this row)	PASS
Every kept order line has a unit_cost (estimated where no product cost exists)	PASS

7. Known limitations (stated honestly; figures refer to the supplied dataset)

• Order dates repaired from shipped_date - 1 day can be off by up to 1 day (shipped - order is 0, 1 or 2 days on every valid order); monthly totals are unaffected except at month boundaries (about 0.07% of NMV moves month).
• Unit prices solved from order_total are exact when the order total is valid; 29 lines fall back to list price x median ratio. Orders with a second unknown line use list price.
• Missing or corrupt cost_price (about 7% of products, cost_imputed = 1) is imputed from the category median cost/list ratio, so Margin carries estimation error for those products (see section 5 for the size). Lines of orphan products get an estimated unit_cost (cost_source = NO_PRODUCT_COST_ESTIMATE).
• Split payments: an order paid in two parts has two payment rows. FactSales takes the is_primary row (Paid/Refunded first, then the larger part), so PaymentMethod shows the larger part's method for orders whose two parts use different methods.
• Shipping fee is imputed with the mean for about 6% of shipments, so total shipping cost is approximate. shipping_days is NULL where delivered or shipped date is missing or was corrupt.
• Province of 25 cities was inferred from coordinates (about 87% leave-one-out accuracy). Gender, seller tier and sub-category stay Unknown where they cannot be inferred.
• Refund amounts of 0 or above the line gross on valid returns are set to the full line amount (modal behaviour). Invalid return dates are set to order date + median lag.
• Customer merging links records by identical email or phone only. Two different people sharing a phone number with dissimilar names are not merged.

8. Key assumptions (the decisions that move the headline numbers)

Each block states the rule, why it was chosen over the alternative, and its measured effect in this run (numbers come from dq_log and sensitivity.csv).

A. Impossible order dates are repaired, not rejected

Rule: an order date that cannot be parsed or lies outside 2024-01-01..2025-06-30 is rebuilt as shipped_date minus the usual lag (1,455 orders), else payment_date minus its lag (137 orders); only 13 orders with neither date are rejected (BAD_DATE / DATE_OUT_OF_RANGE). The lag is learnt from valid rows (shipped - order is 0-2 days on 100% of them, median shown in the decision table).
Why: rejecting those orders would remove 1.03% of NMV (beyond the 1% tolerance) and punch holes in the monthly trend. **Risk:** a repaired date can be 1 day off; about 0.07% of NMV would change month under the alternative anchor.

B. Duplicate customers are merged (email or phone, never name alone)

Rule: records sharing a normalised email, or a phone number plus similar name (>= 0.7), get the lowest customer_id as master; every duplicate-to-master pair is kept in customer_map. 45,000 customer records become 42,444.
Effect: Active Customers 32,836 (unmerged) vs 31,779 (merged), -3.2%. Revenue is unaffected; repeat purchase rate, CLV and RFM are. Without merging, customers are over-counted and repeat buying is understated.

C. Lines of unknown products stay in the data; their cost is estimated

Rule: 932 order lines point to product ids that do not exist. They are kept (revenue is real) and mapped to the Unknown product. Their cost has no product record, so unit_cost = unit_price x 0.572 (median cost/price of lines with a genuine cost), flagged cost_source = NO_PRODUCT_COST_ESTIMATE.
Effect: dropping these lines would lower NMV by 0.36%; costing them at zero would raise margin by 0.22 pt; excluding them changes it by -0.04 pt. All inside the 0.5 pt tolerance.

D. cost_price above list_price is treated as corrupt and imputed

Rule: a product cost that is missing, not positive, or above list price is replaced by list_price x the median cost/list ratio of its category (0.61); flagged cost_imputed = 1.
Evidence: All 65 such products (list price untouched) have cost = 1.50 x list_price, while normal products never exceed 0.97 -> systematic corruption. Keeping those costs would lower margin by 2.00 pt (beyond tolerance), so this decision matters. Imputation quality: imputation error on products with a genuine cost: median 5.4%, p90 13.9%; products with imputed cost carry about 6.9% of lines, and a plain +/-5% bias moves margin only 0.26 pt.